Stage 7. Allocation and tenants

Northlake Town Center, Cedar Row sub-metering, WUI point families, reconciliation and Ridgeline Wind's generated statement

PREPARED BY	Energy Hippo implementation team
STATUS	Ready to enter

Stage 7 builds the new mixed-use Northlake Town Center company and its hierarchy, then exercises every Bill Allocation method, every WUI point-allocation method and every bill generation type the product has, plus the Thermal Plant's internal cost recovery and Cedar Row's residential sub-metering. It is the tenant-rebilling stage: two related but different features share the name "Tenant Rebilling" in the product — Bill Allocation (splitting a parent bill's dollar cost across tenants) and WUI point allocation (splitting a master point's measured use across sub-points) — and this stage uses both, on top of Bill Generation (rate times meter use), which the earlier rate and meter stages already stood up.

Added 2026-10-01: Ridgeline Wind. The Thermal Plant bought a ten-turbine wind company on July 1, 2024, and Stage 7 gains Step 8 and one more Run Now in Step 7. Ridgeline is Bill Generation the other way round from the tenants: no tenant and no allocation, one account (SYN-RWL-G-0000001) whose monthly internal transfer statement EEM writes itself, priced at $0.058 a kWh on the ten turbines less a $1,250.00 monthly research credit. EEM generates a bill per meter, not per account (finding 66), so Stage 2 carries one logical billing meter whose one point is the farm aggregate of the ten turbines, and this step generates from it. The first statement, July 2024, must equal the hand-made one to the cent, $129,330.27. Steps 1 through 6 take nothing from Ridgeline.

Before you start

- Stages 1 through 6 are green. This stage adds a new company and hierarchy of its own (Town Center) and adds records to Northlake University and Cedar Row Apartments, both of which must already exist (companies 6 and 3).
- Two catalog gaps must be closed before typing this stage, and neither is customer data: `dbo.BillAllocationMethod` has only method 1 seeded on this instance today (methods 2-7 are confirmed present in the reference EEMSuite database) and `dbo.PtAllocationMethod` has only 2 of its 3 rows (Percentage Based is missing). Install both, the same way earlier stages treated other ".bak, not in git" catalogs as product install data. Step 4 and Step 5 name the exact rows.
- Enter everything through the applications. If a screen cannot take a value this stage needs, stop and record it under Findings at the end of this chapter and in product findings. Do not fill the gap with SQL.
- Step 8 also needs Ridgeline Wind in place: its Stage 2 account, meters (the billing meter included) and farm aggregate, Stage 3's 10-minute turbine data and aggregation, and Stage 4's rate RWL-XFER-2024 with its determinants under the class All, their commodity mapping and its assignment. The step lists the order.

Step 1. Northlake Town Center company

Where: Database Administrator, Nodes (the company node's Definitions tab); Users & Groups.

Enter: 01-town-center-company.yaml.

Order inside the editor: same as Stage 1 step 1 — Definitions first, save, then Levels, Company Address, Company Contact and Bill Scenarios. Then create the two groups and the one user under Users & Groups.

EEM creates for you: the Actual bill scenario and a 1990-2013 fiscal calendar.

Bill Scenarios: add Budget and Bill Meter Reconciliation to Used, alongside Actual — Step 4's anchor tenant rate and Step 7's reconciliation fixture both need a scenario beyond Actual to assign to.

A loose end for Stage 1's screen, not this one: once Town Center is saved, go back to Utility Providers > Valley Electric District > Associated Companies and give Town Center a vendor number.

Record: the Company ID, and the Group ID/User ID (already the keys you typed).

Check: `town_center_company`.

Step 2. Town Center hierarchy

Where: Database Administrator, Nodes (Property, then Buildings, then Meters, then Points); the Add Billing Account dialog, opened from a meter.

Enter: 02-town-center-hierarchy.yaml.

Order inside the tree: the Property node, save; the five Buildings (Market Hall, Office Tower, Campus Annex, Substation Feed, Property Total), save each; then every Meter under its building — save a Billing Account first through Add Billing Account so the meter has one to attach to. Points last, only on meters that carry real interval data.

Two things every tenant space needs even with no physical device: a Meter-level node (Bill Allocation Configuration details point at a node with a billing account, not a bare name) and, for every tenant Step 4's Building Areas method touches, an Area tab value.

Record: the MetaWorldID, MeterID and BillingAccountID for every row.

Check: `town_center_hierarchy`.

Step 3. Thermal Plant chilled water and steam cost recovery

Where: Web, Bill Allocation Configuration (`#/reporting/BillAllocationConfiguration`).

Enter: 03-thermal-plant-allocation.yaml.

Order inside the screen: same as Step 4 below — pick the master meter, commodity, method and effective period, save, then add candidates.

Read this file's notes before typing it. It makes an explicit, flagged assumption about how Northlake Thermal Plant's internal cost recovery reaches Northlake University's buildings, because Stage 2 (written in parallel) had not landed a node shape to build against. Confirm the two master accounts and the six served-building accounts agree with whatever Stage 2 actually created before treating this step as final.

Record: the BillAllocationConfigID for both configs.

Check: `allocation_configs_by_method` (this step's two rows are included in that check).

Step 4. Bill Allocation Configuration, one config per method

Where: Web, Bill Allocation Configuration (`#/reporting/BillAllocationConfiguration`).

Enter: 04-bill-allocation-configs.yaml, 8 configs covering methods 1 through 7.

Order inside the screen: Master Meter, Commodity, Method and Effective From first, save; then Add Allocated Meters through the Eligible Meters picker; then Combine Overhead; then Balance to 100% if a method 1 config's percentages do not already total 100.

What you type versus what the engine computes: only method 1's rows need Allocation Amount / Allocation % / Overhead typed by hand. Methods 2, 3, 4, 5 and 7 compute their own split — from submeter consumption, floor area or usage share — the first time Process Meter Allocations runs (Step 7); add the candidates and leave their percentages at 0.

Method 6, Bill Aggregation, runs backwards: its master, Property Total, is the receiver, not the source, and every other Town Center tenant meter is a candidate feeding it.

Record: the BillAllocationConfigID for every config.

Check: `allocation_configs_by_method`.

Step 5. WUI point families and fail-reason fixtures

Where: Web, Point Family Configuration (`#/reporting/PointFamilyConfiguration`) — a row's action opens the Master Point Configuration modal.

Enter: 05-wui-point-families.yaml: two real point families (wui_market_electric, wui_tower_electric) and a set of deliberately broken fixtures covering all 5 `PtAllocFailReason` values plus the unimplemented Percentage Based method.

Order inside the modal: pick or confirm the master point, then Add Child Points with each child's Allocation Method and Start Date.

Before this step: install the missing `PtAllocationMethod` row (3, Percentage Based) — without it the last fixture cannot even be selected. Read this file's note on `PtAllocationIndex` before entering the two real families: no screen in the current build appears to write that table, which would leave every index value without an entry path; confirm this in the running application and correct the file if a screen does exist.

Record: nothing new; the modal has no separate ID beyond the point IDs already recorded when the points were created.

Check: `wui_families_and_fail_reasons`.

Step 6. Tenant bill generation types and Cedar Row residential allocation

Where: Database Administrator, Nodes (a meter node's Tenant Rebill tab); Web, Bill Allocation Configuration.

Enter: 06-tenant-bill-generation.yaml: 8 Tenant Rebill tab entries (the 4 generation types plus the 4 Cedar Row master meters' reconciliation flag) and 4 Bill Allocation Configuration configs covering Cedar Row's 248 apartment submeters.

Order inside the Tenant Rebill tab: pick the Bill Scenario at the top, add a Rate Configuration row and save, then switch to Bill Generation and add its row (Assigned Rates, Fiscal Start/End, Schedule, Status, Available %, Offset).

Cedar Row is the same screen and method as Step 4's Tower-Main config, at real scale: method 2, Fully Submetered, one config per building per commodity (4 total), each apartment's own submeter a candidate, no percentage typed anywhere — the engine computes 248 shares when Step 7 runs. House submeters are not candidates; type Overhead % = 2 into every apartment row so their cost is recovered instead of dropped.

Record: nothing new on the Tenant Rebill tab; the 4 Cedar Row BillAllocationConfigIDs.

Check: `generation_configs`, `allocation_configs_by_method` (Cedar Row's 4 configs), and — once Step 7 has run — `tenant_bills_sum_to_parent`.

Step 7. Allocation runs, scheduled tasks and reconciliation

Where: Web, Process Meter Allocations (`#/reporting/ProcessMeterAllocations`); Web, Task Manager (`#/tasks/manager`); Database Administrator, a meter's Tenant Rebill tab for the reconciliation fixture.

Enter: 07-allocation-runs-and-reconciliation.yaml.

Order: run Process Meter Allocations for 2024-01 through 2024-03 first (it needs Step 5's families saved). Then schedule and Run Now Task_Tbl 24 (Process Bill Allocation) and 42 (Run Bill Gen) from Task Manager — Task_Tbl 47 (Run WUI Process) is system-only and needs no schedule of its own. Then enter the Anchor Store reconciliation fixture on its Tenant Rebill tab. Run Now on Run Bill Gen once more after Step 8, once Ridgeline's data is in.

Tenant accruals have no entry path at all — no UI, no API, no scheduled task writes `BillAccrualMap`. This step does not attempt one; see Findings.

Record: nothing generated; this step produces data, not IDs.

Check: `allocation_runs`, `tenant_bills_sum_to_parent`, `reconciliation_variance`.

Step 8. Ridgeline Wind transfer statement

Where: Database Administrator, Nodes (a meter node's Tenant Rebill tab); Web, Generated Bills (`#/reporting/GeneratedBills`) to watch it.

Enter: 08-ridgeline-transfer-generation.yaml: the Bill Generation row for Stage 2's billing meter, with the acceptance case, the WTG-07 outage months and the run order.

Bill Generation with no tenant. The Thermal Plant buys its own wind farm's output at a transfer price and nobody sends a bill: EEM writes the monthly statement for account SYN-RWL-G-0000001 from the ten turbines' Generated kWh 10m points and rate RWL-XFER-2024 ($0.058 a kWh and a $1,250.00 monthly research credit).

The first statement, July 2024, must equal Ridgeline accounting's hand-made one to the cent: 2,251,384 kWh, $130,580.27 of energy, less the credit, $129,330.27.

Why a billing meter and not ten turbine meters. Generation prices the points under one meter node and writes one bill per meter per month (finding 66). On the ten turbine meters it would write ten bills a month, each repeating the credit. So Stage 2 carries a billing meter, Ridgeline Farm Transfer Meter (SYN-RWL-M-0110000), under Ridgeline Turbine Field, whose one point is the farm aggregate Ridgeline Farm Generated kWh 10m, the way EEM's own demo database bills from aggregates, and this step generates from it. The turbine meters keep their account and get neither a rate nor a generation row.

Order inside the tab: the meter must already have its aggregate (the Bill Generation row cannot be saved on a meter with no point), and Stage 4 has assigned RWL-XFER-2024 to it with Bill unchecked. This step adds only the Bill Generation row: Company Calendar, Auto, 2024-07 to 2025-12, Available % 100, Offset 0. A company administrator enters it, because the save refuses anyone else.

Run order: Stage 3's turbine data and aggregation, Stage 4's rate (determinants under All, commodity mapping, assignment), then this step. The Billing Charges report on the billing meter previews July before any bill is written; then Run Now on Run Bill Gen (Step 7) writes one bill per complete month. A month below Available % waits; it is not an error. Two months are short for clock reasons. The engine cuts months on the points' stored Pacific time while the SCADA clock stays on PST (finding 46), so July 2024 starts an hour before the SCADA's own July: Stage 3's feed must begin with the row stamped 2024-06-30 23:10 PST (the ten minutes from Pacific midnight) or July is an hour short, waits, and the acceptance case cannot pass. March 2025 holds 4,458 of 4,464 whatever the feed, because Pacific time skips an hour on 2025-03-09: set Available % to 99, Run Now once, and set it back to 100 (assumed; a manual Create needs Bill Entry Web, which no Thermal Plant user holds). The engine reads a stamp as the end of its interval, so a month takes stamps after 00:00 on the 1st up to and including 00:00 on the next 1st. The SCADA file is written with interval-end stamps and Stage 3 cuts its sums the same way, so a feed through the row stamped 2026-01-01 00:00 PST is exactly July 2024 to December 2025.

What to expect: 18 statements, 2024-07 to 2025-12, each with the transfer-energy line and the credit once. WTG-07's gearbox outage (2025-02-11 to 2025-04-22) makes February to April lower with no adjustment line: the turbine keeps reporting zero energy, so it is lost production, not missing data. The statement is dated its period end, not 2024-08-02, and has no invoice number, due date or the four informational rows of the hand-made one; its two lines and total must match.

Record: nothing new; the billing meter's IDs are Stage 2's.

Check: `ridgeline_billing_meter`, `ridgeline_commodity_mapping`, `ridgeline_generation_config`, `ridgeline_generation_log`, `ridgeline_generated_statements`, `ridgeline_acceptance_2024_07`, `ridgeline_outage_months`.

Stage 7 is done when

- Northlake Town Center exists with its levels, address, contact, two groups and one user, and its five buildings, 22 meters and billing accounts, and area values on every tenant space Method 4 and the WUI fixtures need.
- Every Bill Allocation method 1 through 7 has at least one config that produces tenant bills, plus the Thermal Plant's chilled-water and steam recovery and Cedar Row's 4 residential configs.

- Every allocated child's amount, including overhead, sums to its parent bill to the penny — `tenant_bills_sum_to_parent` shows Diff = 0 on every row and Property Total equal to the sum of the bills it aggregates.
- Both WUI point families produce allocated use proportional to index, area and days, and each of `PtAllocFailReason` 1 through 5 has been produced by its fixture at least once; the Percentage Based fixture produces neither a success nor a failure row.
- All 4 bill generation types behave as configured, and the 3 Task_Tbl tasks this stage depends on are confirmed present and, for 24 and 42, scheduled.
- `rp_BillReconciliation` shows a nonzero variance for Anchor Store in 2024-01, and every other Tenant Rebilling report this instance's catalog carries renders without error.
- Ridgeline's account SYN-RWL-G-0000001 holds 18 generated statements, one a month from July 2024 to December 2025, each with the transfer-energy line and the research credit once. July 2024 reads $129,330.27 and its per-turbine sums match the hand-made statement (`ridgeline_acceptance_2024_07`, Diff 0), and February to April 2025 are lower with no adjustment line.

Run `checks.sql` beside this chapter against the instance database to see each of these as a result set.

Findings

Product gaps and open questions found while building this stage, in addition to what earlier stages already logged in product findings:

- **Two catalog gaps, not customer data.** This instance's `BillAllocationMethod` table carries only method 1 (of 7) and `PtAllocationMethod` carries only 2 (of 3, missing Percentage Based). Both are confirmed present with the expected names, descriptions and procs in the reference EEMSuite database, and both fit the precedent already established for other install-time catalogs: standing them up is finishing the install, not seeding this customer's data. Neither can be typed around from a screen.
- **Three Tenant Rebilling reports are not registered on this instance.** `dbo.GuiReports_Tbl` has 11 of the 14 rows the reference database carries for this feature; missing are `rp_accruals`, `rp_mpaaccruals` and `rp_OpenAccruals`, all three accrual reports. They will not appear on the Reporting menu, with data or without, until their catalog rows are installed — consistent with the gap below.
- **Tenant accruals (product-gap-backlog #4, "Accruals create/reverse feature") have no entry path.** No screen, API or scheduled task writes `dbo.BillAccrualMap`; `rp_OpenAccruals @Reverse` exists as a proc but nothing calls it from the UI. This blocks Phase 4's tenant accrual case entirely, not just this stage's demo of it.
- **Percentage Based (`PtAllocationMethod` 3) is unimplemented in the engine, confirmed again here.** Once its catalog row is installed it can be selected on the Master Point Configuration modal, but a child configured with it produces no allocated use and no `PtAllocFailReason` row — it is silently ignored. This matches the Phase 4 spec's documented expectation exactly.
- `PtAllocationIndex` **appears to have no entry screen.** The modal that configures WUI child points (Master Point Configuration) has no index field, and no C# source under `src/Domain` or `src/Application` references `dbo.PtAllocationIndex` by name, even though `Database/EEMSuite/Procs/WUIPtAllocationIndex_Save.sql` exists and is callable. If that is confirmed by running the application rather than reading its source, it is a missed-screen finding: the backend and the proc exist, but nothing in the current build calls it.

- **Per-meter rate-class/option binding (product-gap-backlog #5, "Per-meter rate-class / option binding") still has no field on the Tenant Rebill tab's Rate Configuration grid.** Not exercised by this stage directly, but every tenant meter here is exactly the kind of assignment that gap affects; carried forward from the backlog being dissolved into this chapter set.
- **The Thermal Plant's cost-recovery mechanism (Step 3) is this chapter's own assumption, not a confirmed product fact**, because it depends on how Stage 2 shapes Northlake University's served buildings, and Stage 2 was still being written in parallel. Confirm the master/served-building structure once Stage 2 lands, and adjust Step 3 if it does not match.
- **Bill Generation cannot bill a sum across meters (finding 66, Ridgeline).** `UtilityRates_GetPoints` reads only the points under the one meter node, and `UtilityGenBill_Create` writes one bill per meter and month. The ten turbine meters would give ten bills a month, each repeating the monthly credit (July 2024's lines would add to $118,080.28, not $129,330.27; even the ten energy lines alone add to $130,580.28, a cent over the farm line). The workaround is the one EEM's demo database uses: a logical billing meter whose one point is an aggregate, which Stage 2 now carries (Ridgeline Farm Transfer Meter, with the farm aggregate under it). `ridgeline_generated_statements` confirms it: one bill a month on the account.
- **A determinant under an added rate class may price nothing on a generated bill (finding 69; read from the code, not yet seen on a run).** `UtilityRate_Exec` takes a rate class only if it is the All type, or its rate option is the default, or the meter is bound to it. Add Rate creates the class All with that type, but every class added after it is Unique and no screen binds a meter to one (findings 7 and 8), so a determinant under an added class prices only as its option's default. Rate tests pass regardless, because they name the classes. RWL-XFER-2024 keeps both determinants under All.
- **Offset cannot hold a statement back, and a generated bill carries no issue date (finding 70).** A period is listed once its end is before today plus Offset, so a positive Offset brings a month forward, and the desktop clamps Offset at 0. A generated bill is dated its period end, has no invoice number or due date, and is written by the task's login. The two-day entry lag and the keiko.tan entry user planned for Ridgeline therefore have no setting.
- **An unmapped determinant drops its line, and generated-bill bookkeeping is loose (findings 71 and 72).** A determinant with no commodity mapping stays in the bill total and writes no line, so the lines stop adding to the total (`ridgeline_commodity_mapping` and `ridgeline_generated_statements` catch it). The bill total keeps four decimals while each line rounds to the cent (July 2024: 129,330.2720 against lines adding to 129,330.27), so compare `ROUND(BL_Amount, 2)`; the day count is end minus start (30 for July); and the availability test counts every non-excluded point's rows against one point's expected intervals and counts a daylight-saving gap as missing data.

If a screen refuses any other value while entering this stage, add it here with the screen, the field and what happened.